the file contains 50 algorithm tasks create pseudocode for each and display results as a numbered list

Part 1 – Pseudocode for Tasks 1–10

1. Determine whether two given strings are anagrams of each other.

```
BEGIN
    INPUT String1, String2

    IF LENGTH(String1) ≠ LENGTH(String2) THEN
        OUTPUT "Not Anagrams"
    ELSE
        SORT characters of String1
        SORT characters of String2

        IF String1 = String2 THEN
            OUTPUT "Anagrams"
        ELSE
            OUTPUT "Not Anagrams"
        ENDIF
    ENDIF
END
```

---

2. Convert a non-negative decimal integer into its binary string representation.

```
BEGIN
    INPUT Number
    Binary ← ""

    IF Number = 0 THEN
        OUTPUT "0"
    ELSE
        WHILE Number > 0
            Remainder ← Number MOD 2
            Binary ← Remainder + Binary
            Number ← Number DIV 2
        ENDWHILE

        OUTPUT Binary
    ENDIF
END
```

---

3. Determine whether a given year satisfies the leap year rules.

```
BEGIN
    INPUT Year

    IF (Year MOD 400 = 0) THEN
        OUTPUT "Leap Year"
    ELSE IF (Year MOD 100 = 0) THEN
        OUTPUT "Not Leap Year"
    ELSE IF (Year MOD 4 = 0) THEN
        OUTPUT "Leap Year"
    ELSE
        OUTPUT "Not Leap Year"
    ENDIF
END
```

---

4. Find the intersection of two sorted arrays.

```
BEGIN
    INPUT Array1, Array2

    i ← 0
    j ← 0

    WHILE i < LENGTH(Array1) AND j < LENGTH(Array2)
        IF Array1[i] = Array2[j] THEN
            OUTPUT Array1[i]
            i ← i + 1
            j ← j + 1
        ELSE IF Array1[i] < Array2[j] THEN
            i ← i + 1
        ELSE
            j ← j + 1
        ENDIF
    ENDWHILE
END
```

---

5. Remove punctuation from a sentence.

```
BEGIN
    INPUT Sentence
    Result ← ""

    FOR EACH Character IN Sentence
        IF Character is NOT punctuation THEN
            Result ← Result + Character
        ENDIF
    ENDFOR

    OUTPUT Result
END
```

---

6. Find the length of a given string.

```
BEGIN
    INPUT Text
    Count ← 0

    FOR EACH Character IN Text
        Count ← Count + 1
    ENDFOR

    OUTPUT Count
END
```

---

7. Split a sentence into individual words and store them in a list.

```
BEGIN
    INPUT Sentence

    Words ← SPLIT Sentence BY spaces

    OUTPUT Words
END
```

---

8. Sort a list of words in alphabetical order.

```
BEGIN
    INPUT WordList

    FOR i ← 0 TO LENGTH(WordList)-2
        FOR j ← 0 TO LENGTH(WordList)-2-i
            IF WordList[j] > WordList[j+1] THEN
                SWAP WordList[j], WordList[j+1]
            ENDIF
        ENDFOR
    ENDFOR

    OUTPUT WordList
END
```

---

9. Find the exact position of a target value in an unsorted list.

```
BEGIN
    INPUT List, Target

    Found ← FALSE

    FOR i ← 0 TO LENGTH(List)-1
        IF List[i] = Target THEN
            OUTPUT "Found at position", i
            Found ← TRUE
            EXIT FOR
        ENDIF
    ENDFOR

    IF Found = FALSE THEN
        OUTPUT "Target not found"
    ENDIF
END
```

---

10. Count how many times each vowel appears in a paragraph.

```
BEGIN
    INPUT Text

    A ← 0
    E ← 0
    I ← 0
    O ← 0
    U ← 0

    FOR EACH Character IN LOWERCASE(Text)
        CASE Character OF
            'a': A ← A + 1
            'e': E ← E + 1
            'i': I ← I + 1
            'o': O ← O + 1
            'u': U ← U + 1
        ENDCASE
    ENDFOR

    OUTPUT A, E, I, O, U
END
```

Part 2 – Pseudocode for Tasks 11–20

11. Remove duplicate IP addresses and return only unique IP addresses.

```
BEGIN
    INPUT IP_List

    UniqueIPs ← Empty Set

    FOR EACH IP IN IP_List
        ADD IP TO UniqueIPs
    ENDFOR

    OUTPUT UniqueIPs
END
```

---

12. Calculate the sum of the main diagonal of an N × N matrix.

```
BEGIN
    INPUT Matrix, N

    Sum ← 0

    FOR i ← 0 TO N - 1
        Sum ← Sum + Matrix[i][i]
    ENDFOR

    OUTPUT Sum
END
```

---

13. Reverse a string without using built-in reverse functions.

```
BEGIN
    INPUT Text

    Reverse ← ""

    FOR i ← LENGTH(Text)-1 DOWNTO 0
        Reverse ← Reverse + Text[i]
    ENDFOR

    OUTPUT Reverse
END
```

---

14. Find the sum of all even numbers in a list.

```
BEGIN
    INPUT Numbers

    Sum ← 0

    FOR EACH Number IN Numbers
        IF Number MOD 2 = 0 THEN
            Sum ← Sum + Number
        ENDIF
    ENDFOR

    OUTPUT Sum
END
```

15. Find the second largest number in an array.

```
BEGIN
    INPUT Array

    Largest ← -∞
    SecondLargest ← -∞

    FOR EACH Number IN Array
        IF Number > Largest THEN
            SecondLargest ← Largest
            Largest ← Number
        ELSE IF Number > SecondLargest AND Number < Largest THEN
            SecondLargest ← Number
        ENDIF
    ENDFOR

    OUTPUT SecondLargest
END
```

---

16. Find the GCD of two numbers using the Euclidean algorithm.

```
BEGIN
    INPUT A, B

    WHILE B ≠ 0
        Temp ← B
        B ← A MOD B
        A ← Temp
    ENDWHILE

    OUTPUT A
END
```

---

17. Compute the sum and average of a list of numbers.

```
BEGIN
    INPUT Numbers

    Sum ← 0

    FOR EACH Number IN Numbers
        Sum ← Sum + Number
    ENDFOR

    Average ← Sum / LENGTH(Numbers)

    OUTPUT Sum
    OUTPUT Average
END
```

---

18. Count the frequency of each character in a string.

```
BEGIN
    INPUT Text

    Create empty Dictionary
```

```
    FOR EACH Character IN Text
        IF Character EXISTS in Dictionary THEN
            Dictionary[Character] ← Dictionary[Character] + 1
        ELSE
            Dictionary[Character] ← 1
        ENDIF
    ENDFOR

    OUTPUT Dictionary
END
```

---

19. Swap two variables without using a temporary variable.

```
BEGIN
    INPUT A, B

    A ← A + B
    B ← A - B
    A ← A - B

    OUTPUT A, B
END
```

---

20. Determine all divisors of a positive integer.

```
BEGIN
    INPUT Number

    FOR i ← 1 TO Number
        IF Number MOD i = 0 THEN
            OUTPUT i
        ENDIF
    ENDFOR
END
```

Part 3 – Pseudocode for Tasks 21–30

21. Determine whether a number is even or odd.

```
BEGIN
    INPUT Number

    IF Number MOD 2 = 0 THEN
        OUTPUT "Even"
    ELSE
        OUTPUT "Odd"
    ENDIF
END
```

---

22. Compute the factorial of a positive integer.

```
BEGIN
    INPUT Number

    Factorial ← 1

    FOR i ← 1 TO Number
        Factorial ← Factorial × i
```

```
    ENDFOR

    OUTPUT Factorial
END
```

---

23. Convert numerical scores into grade categories (Pass, Merit, Distinction).

```
BEGIN
    INPUT Score

    IF Score >= 70 THEN
        OUTPUT "Distinction"
    ELSE IF Score >= 50 THEN
        OUTPUT "Merit"
    ELSE IF Score >= 40 THEN
        OUTPUT "Pass"
    ELSE
        OUTPUT "Fail"
    ENDIF
END
```

---

24. Check whether a password contains at least 8 characters, one number, and one uppercase letter.

```
BEGIN
    INPUT Password

    HasNumber ← FALSE
    HasUpper ← FALSE

    IF LENGTH(Password) < 8 THEN
        OUTPUT "Invalid Password"
    ELSE
        FOR EACH Character IN Password
            IF Character is a digit THEN
                HasNumber ← TRUE
            ENDIF

            IF Character is an uppercase letter THEN
                HasUpper ← TRUE
            ENDIF
        ENDFOR

        IF HasNumber = TRUE AND HasUpper = TRUE THEN
            OUTPUT "Valid Password"
        ELSE
            OUTPUT "Invalid Password"
        ENDIF
    ENDIF
END
```

---

25. Find the length of the longest substring without repeating characters.

```
BEGIN
    INPUT Text

    MaxLength ← 0

    FOR Start ← 0 TO LENGTH(Text)-1
        Seen ← Empty Set
        CurrentLength ← 0
```

```
        FOR End ← Start TO LENGTH(Text)-1
            IF Text[End] EXISTS IN Seen THEN
                EXIT FOR
            ENDIF

            ADD Text[End] TO Seen
            CurrentLength ← CurrentLength + 1
        ENDFOR

        IF CurrentLength > MaxLength THEN
            MaxLength ← CurrentLength
        ENDIF
    ENDFOR

    OUTPUT MaxLength
END
```

---

26. Perform level-order traversal on a binary tree.

```
BEGIN
    INPUT Root

    CREATE Empty Queue
    ENQUEUE Root

    WHILE Queue is not empty
        Node ← DEQUEUE Queue
        OUTPUT Node.Value

        IF Node.Left exists THEN
            ENQUEUE Node.Left
        ENDIF

        IF Node.Right exists THEN
            ENQUEUE Node.Right
        ENDIF
    ENDWHILE
END
```

---

27. Rotate a square matrix 90° clockwise in-place.

```
BEGIN
    INPUT Matrix, N

    TRANSPOSE Matrix

    FOR EACH Row IN Matrix
        REVERSE Row
    ENDFOR

    OUTPUT Matrix
END
```

---

28. Return the maximum value in every sliding window.

```
BEGIN
    INPUT Array, WindowSize

    FOR i ← 0 TO LENGTH(Array) - WindowSize
        Maximum ← Array[i]
```

```
        FOR j ← i TO i + WindowSize - 1
            IF Array[j] > Maximum THEN
                Maximum ← Array[j]
            ENDIF
        ENDFOR

        OUTPUT Maximum
    ENDFOR
END
```

---

29. Build a text classification model to categorise text.

```
BEGIN
    LOAD TrainingData

    PREPROCESS text
    EXTRACT text features

    TRAIN ClassificationModel

    INPUT NewText

    PREPROCESS NewText
    EXTRACT features

    Category ← ClassificationModel.PREDICT(NewText)

    OUTPUT Category
END
```

---

30. Create a topic modelling system to extract topics from a corpus.

```
BEGIN
    LOAD Documents

    PREPROCESS Documents

    CREATE Document-Term Matrix

    APPLY Topic Modelling Algorithm

    FOR EACH Topic
        OUTPUT Topic Keywords
    ENDFOR
END
```

Part 4 – Pseudocode for Tasks 31–40

31. Develop a co-reference resolution system to identify relationships between pronouns and nouns.

```
BEGIN
    INPUT Text

    PREPROCESS Text

    IDENTIFY all nouns
    IDENTIFY all pronouns
```

```
    FOR EACH Pronoun
        FIND the most likely matching noun
        LINK Pronoun to Noun
    ENDFOR

    OUTPUT Linked References
END
```

---

32. Build a question answering system based on a given text.

```
BEGIN
    INPUT Text
    INPUT Question

    PREPROCESS Text
    PREPROCESS Question

    FIND relevant information in Text

    GENERATE Answer

    OUTPUT Answer
END
```

---

33. Find the square root of a non-negative integer using binary search.

```
BEGIN
    INPUT Number

    Low ← 0
    High ← Number
    Answer ← 0

    WHILE Low ≤ High
        Mid ← (Low + High) DIV 2

        IF Mid × Mid = Number THEN
            OUTPUT Mid
            STOP
        ELSE IF Mid × Mid < Number THEN
            Answer ← Mid
            Low ← Mid + 1
        ELSE
            High ← Mid - 1
        ENDIF
    ENDWHILE

    OUTPUT Answer
END
```

---

34. Find all common elements in two unsorted lists without duplicates.

```
BEGIN
    INPUT List1, List2

    Result ← Empty Set

    FOR EACH Item IN List1
        IF Item EXISTS IN List2 THEN
            ADD Item TO Result
        ENDIF
```

```
    ENDFOR

    OUTPUT Result
END
```

---

35. Design a memory buffer that discards the least recently accessed item when full.

```
BEGIN
    CREATE Buffer with fixed capacity

    WHEN Item is accessed
        MARK Item as most recently used
    END

    WHEN New Item is added
        IF Buffer is full THEN
            REMOVE least recently used item
        ENDIF

        INSERT New Item
    END

    OUTPUT Buffer
END
```

---

36. Find a target element in a rotated sorted array.

```
BEGIN
    INPUT Array, Target

    Low ← 0
    High ← LENGTH(Array) - 1

    WHILE Low ≤ High
        Mid ← (Low + High) DIV 2

        IF Array[Mid] = Target THEN
            OUTPUT Mid
            STOP
        ENDIF

        IF Array[Low] ≤ Array[Mid] THEN
            IF Target ≥ Array[Low] AND Target < Array[Mid] THEN
                High ← Mid - 1
            ELSE
                Low ← Mid + 1
            ENDIF
        ELSE
            IF Target > Array[Mid] AND Target ≤ Array[High] THEN
                Low ← Mid + 1
            ELSE
                High ← Mid - 1
            ENDIF
        ENDIF
    ENDWHILE

    OUTPUT "Not Found"
END
```

---

37. Implement a stack using an array.

```
BEGIN
    CREATE Stack
    Top ← -1

    PUSH(Item)
        Top ← Top + 1
        Stack[Top] ← Item

    POP()
        IF Top = -1 THEN
            OUTPUT "Stack Empty"
        ELSE
            Item ← Stack[Top]
            Top ← Top - 1
            OUTPUT Item
        ENDIF

    PEEK()
        IF Top ≠ -1 THEN
            OUTPUT Stack[Top]
        ENDIF
END
```

---

38. Find the first non-repeating character in a string.

```
BEGIN
    INPUT Text

    COUNT frequency of each character

    FOR EACH Character IN Text
        IF Frequency(Character) = 1 THEN
            OUTPUT Character
            STOP
        ENDIF
    ENDFOR

    OUTPUT "None Found"
END
```

---

39. Detect a cycle in a singly linked list (Floyd's Cycle Detection).

```
BEGIN
    INPUT Head

    Slow ← Head
    Fast ← Head

    WHILE Fast ≠ NULL AND Fast.Next ≠ NULL
        Slow ← Slow.Next
        Fast ← Fast.Next.Next

        IF Slow = Fast THEN
            OUTPUT "Cycle Detected"
            STOP
        ENDIF
    ENDWHILE

    OUTPUT "No Cycle"
END
```

---

40. Implement a queue using two stacks.

```
BEGIN
    CREATE Stack1
    CREATE Stack2

    ENQUEUE(Item)
        PUSH Item onto Stack1

    DEQUEUE()
        IF Stack2 is empty THEN
            WHILE Stack1 is not empty
                PUSH POP(Stack1) onto Stack2
            ENDWHILE
        ENDIF

        OUTPUT POP(Stack2)
END
```

Part 5 – Pseudocode for Tasks 41–50

41. Solve the N Queens Problem for an arbitrary board size N.

```
BEGIN
    INPUT N

    CREATE N × N board

    FUNCTION Solve(Row)
        IF Row = N THEN
            OUTPUT Board
            RETURN
        ENDIF

        FOR Col ← 0 TO N-1
            IF Position(Row, Col) is safe THEN
                PLACE Queen at (Row, Col)

                Solve(Row + 1)

                REMOVE Queen from (Row, Col)
            ENDIF
        ENDFOR
    END FUNCTION

    Solve(0)
END
```

---

42. Find all pairs in an array that sum to a given target value.

```
BEGIN
    INPUT Array, Target

    FOR i ← 0 TO LENGTH(Array)-2
        FOR j ← i+1 TO LENGTH(Array)-1
            IF Array[i] + Array[j] = Target THEN
                OUTPUT Array[i], Array[j]
            ENDIF
        ENDFOR
    ENDFOR
END
```

---

43. Solve the Tower of Hanoi problem for N discs.

```
BEGIN
    INPUT N

    FUNCTION Hanoi(Discs, Source, Auxiliary, Destination)
        IF Discs = 1 THEN
            OUTPUT "Move disk from", Source, "to", Destination
        ELSE
            Hanoi(Discs-1, Source, Destination, Auxiliary)

            OUTPUT "Move disk from", Source, "to", Destination

            Hanoi(Discs-1, Auxiliary, Source, Destination)
        ENDIF
    END FUNCTION

    Hanoi(N, "A", "B", "C")
END
```

---

44. Find the contiguous subarray with the maximum sum (Kadane's Algorithm).

```
BEGIN
    INPUT Array

    CurrentSum ← Array[0]
    MaxSum ← Array[0]

    FOR i ← 1 TO LENGTH(Array)-1
        CurrentSum ← MAX(Array[i], CurrentSum + Array[i])

        IF CurrentSum > MaxSum THEN
            MaxSum ← CurrentSum
        ENDIF
    ENDFOR

    OUTPUT MaxSum
END
```

---

45. Create instructions for adding two matrices of equal size.

```
BEGIN
    INPUT MatrixA, MatrixB
    INPUT Rows, Columns

    FOR i ← 0 TO Rows-1
        FOR j ← 0 TO Columns-1
            Result[i][j] ← MatrixA[i][j] + MatrixB[i][j]
        ENDFOR
    ENDFOR

    OUTPUT Result
END
```

---

46. Develop a process for multiplying two matrices.

```
BEGIN
    INPUT MatrixA, MatrixB
    INPUT RowsA, ColumnsA, ColumnsB

    FOR i ← 0 TO RowsA-1
```

```
        FOR j ← 0 TO ColumnsB-1
            Result[i][j] ← 0

            FOR k ← 0 TO ColumnsA-1
                Result[i][j] ← Result[i][j] +
                                MatrixA[i][k] × MatrixB[k][j]
            ENDFOR
        ENDFOR
    ENDFOR

    OUTPUT Result
END
```

---

47. Update inventory quantities after purchases and deliveries.

```
BEGIN
    INPUT Inventory
    INPUT TransactionType
    INPUT Item
    INPUT Quantity

    IF TransactionType = "Purchase" THEN
        Inventory[Item] ← Inventory[Item] - Quantity

    ELSE IF TransactionType = "Delivery" THEN
        Inventory[Item] ← Inventory[Item] + Quantity
    ENDIF

    OUTPUT Inventory
END
```

---

48. Search through nested folders recursively to locate a specific file type.

```
BEGIN
    INPUT Folder
    INPUT FileType

    FUNCTION Search(CurrentFolder)
        FOR EACH Item IN CurrentFolder
            IF Item is a file AND Item.Type = FileType THEN
                OUTPUT Item
            ELSE IF Item is a folder THEN
                Search(Item)
            ENDIF
        ENDFOR
    END FUNCTION

    Search(Folder)
END
```

---

49. Create a pathfinding solution through a maze using graph traversal.

```
BEGIN
    INPUT Maze
    INPUT Start, Goal

    CREATE Queue
    MARK Start as visited
    ENQUEUE Start

    WHILE Queue is not empty
```

```
        Current ← DEQUEUE Queue

        IF Current = Goal THEN
            OUTPUT "Path Found"
            STOP
        ENDIF

        FOR EACH Neighbour of Current
            IF Neighbour not visited THEN
                MARK Neighbour as visited
                ENQUEUE Neighbour
            ENDIF
        ENDFOR
    ENDWHILE

    OUTPUT "No Path Found"
END
```

---

50. Develop algorithms enabling autonomous drones to coordinate movement while avoiding collisions.

```
BEGIN
    INPUT DroneList
    INPUT Destination

    WHILE Drones have not reached Destination

        FOR EACH Drone
            DETECT nearby obstacles
            DETECT nearby drones

            CALCULATE safe path

            UPDATE Drone position
        ENDFOR

        CHECK for possible collisions

        IF Collision risk detected THEN
            RECALCULATE paths
        ENDIF

    ENDWHILE

    OUTPUT "All drones reached destination safely"
END
```